



Typical Decoding for Natural Language Generation

딥논워 자연어처리팀 2022.05.15

김수빈, 신문중, 박희수, 조진욱, 진명훈(발표자), 황경진

Contents

- How to generate text?
- Information-Theoretic View of Natural Language
- Typical Decoding
- Experiments & Results
- Conclusion

2022.02.01

Typical Decoding for Natural Language Generation

Clara Meister¹ Tiago Pimentel² Gian Wiher¹ Ryan Cotterell^{1,2}

¹ETH Zürich ²University of Cambridge

clara.meister@inf.ethz.ch tp472@cam.ac.uk

gian.wiher@inf.ethz.ch ryan.cotterell@inf.ethz.ch

How to generate text?

Probabilistic Language Generators

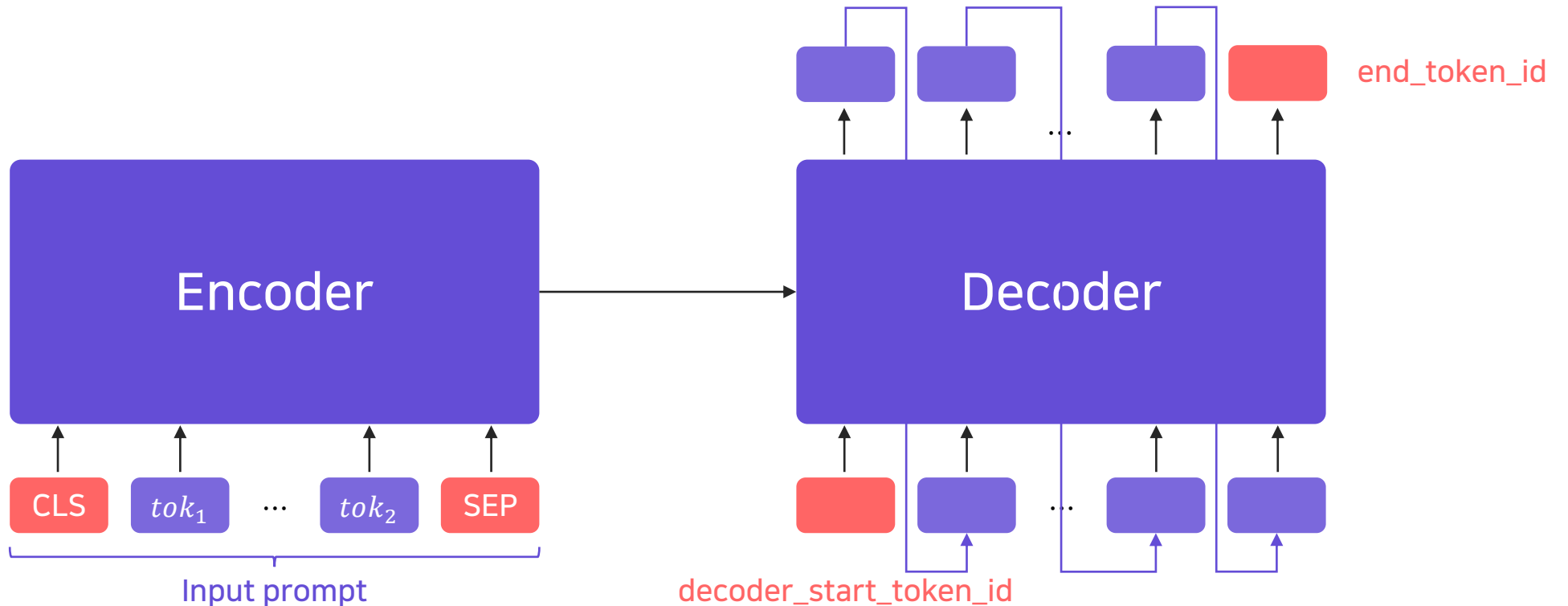
- Natural Language Generation: 자연어 생성
- 언어 모델링, 추상 요약, 스토리 생성, 번역 등 다양한 NLP task에 적용
- 아래와 같은 수식으로 쓸 수 있습니다!

$$q(y) = \prod_{t=1}^{|y|} q(y_t | y_{<t})$$

- q : probability distributions
- $y = \langle y_1, y_2, \dots \rangle$: natural language strings
- 조건부에 input prompt가 추가될 수 있음 (e.g., abstractive summarization)

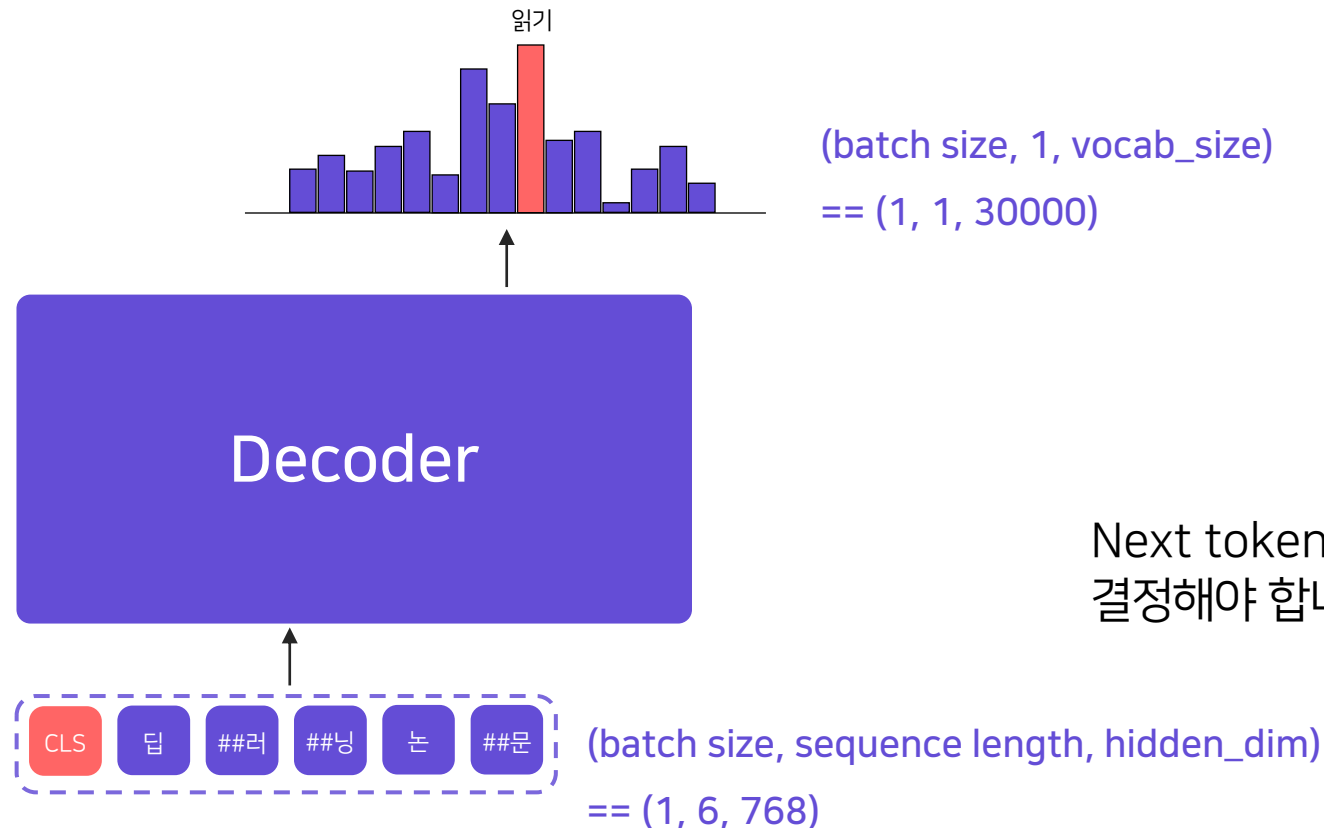
Probabilistic Language Generators

- Seq2Seq (e.g., BART, T5)
- Decoder-only (e.g., GPT)
 - In context learning



Decoding Natural Language Strings

- Text 생성 과정에 대해 조금 더 자세히 알아보자!

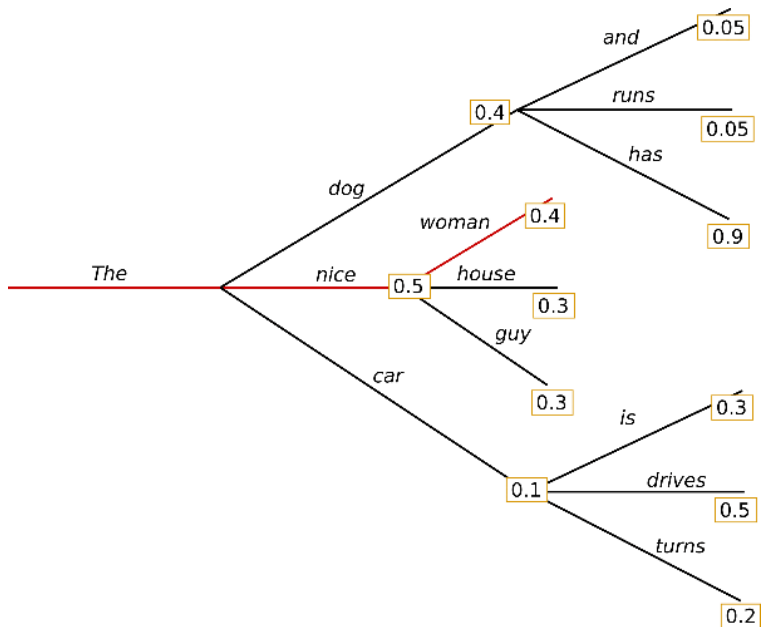


$$q(y) = \prod_{t=1}^{|y|} q(y_t | y_{<t})$$

Next token distribution에서 어떤 문자열을 선택할 지 결정해야 합니다!! → Decision rule

Mode seeking

- Next token distribution에서 어떤 token을 고를 것인가?
- 가장 이상적인 방법은 모든 경우의 수를 고려해서 가장 높은 확률을 가지는 Sequence를 고르는 것!
 - MAP (Maximum a Priori) → 이는 NP-hard 문제입니다! 때문에 heuristic fashion으로 접근하지요
- Greedy decoding (k=1)



```
input_ids = tokenizer.encode("근육이 커지기 위해서는", return_tensors="pt")

# CLM으로 문장을 생성 (output length가 128에 도달할 때 까지)
greedy_output = model.generate(input_ids, max_length=128)

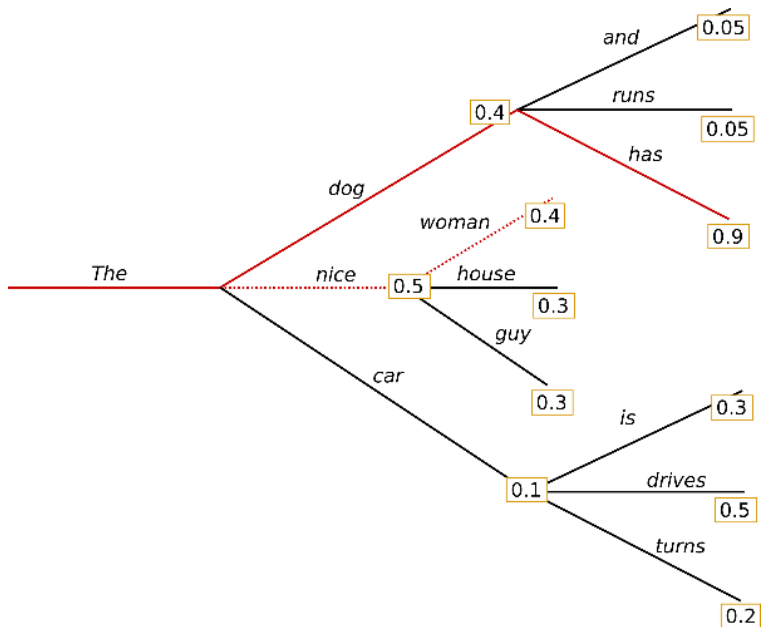
print("Output:\n" + 100 * '-')
print(tokenizer.decode(greedy_output[0], skip_special_tokens=True))
```

Output:

근육이 커지기 위해서는 무엇보다 규칙적인 생활습관이 중요하다.
특히, 아침식사는 단백질과 비타민, 무기질 등 영양소가 풍부한 음식을 골고루 섭취하는 것이 좋다.
또한 규칙적인 운동은 근육을 강화시켜주는 효과가 있다.
특히, 아침식사는 단백질과 비타민, 무기질 등 영양소가 풍부한 음식을 골고루 섭취하는 것이 좋다.
또한 규칙적인 운동은 근육을 강화시켜주는 효과가 있다.
특히, 아침식사는 단백질과 비타민, 무기질 등 영양소가 풍부한 음식을 골고루 섭취하는 것이 좋다.
또한 규칙적인 운동은 근육을 강화시켜주는 효과가 있다.
근육을 강화시켜주는 운동은 근육을 강화시켜주는 효과가 있다.
근육을 강화시켜주는 운동은 근육을 강화

Mode seeking

- Next token distribution에서 어떤 token을 고를 것인가?
- 가장 이상적인 방법은 모든 경우의 수를 고려해서 가장 높은 확률을 가지는 Sequence를 고르는 것!
 - MAP (Maximum a Priori) → 이는 NP-hard 문제입니다! 때문에 heuristic fashion으로 접근하지요
- Beam Search ($k \geq 2$)



```
beam_output = model.generate(
    input_ids,
    max_length=128,
    num_beams=5,
    early_stopping=True
)

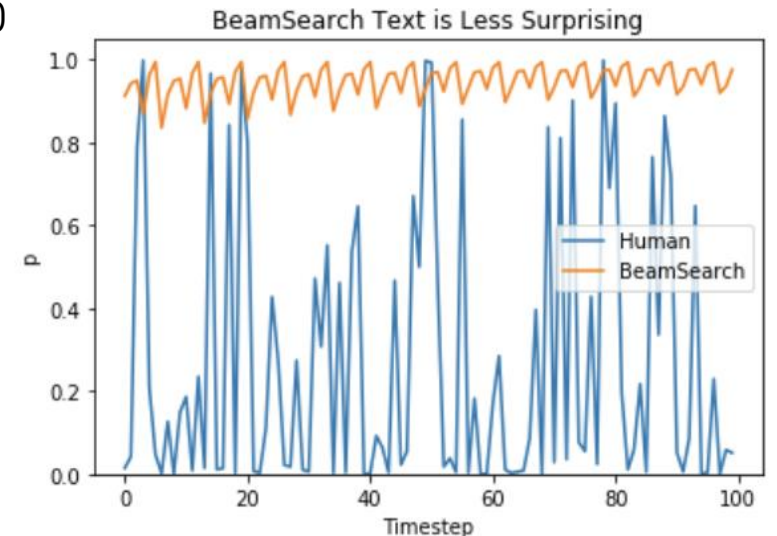
print("Output:\n" + 100 * '-')
print(tokenizer.decode(beam_output[0], skip_special_tokens=True))
```

Output:

근육이 커지기 위해서는 피부 속 콜라겐과 엘라스틴의 생성을 촉진시키는 것이 중요하다.
콜라겐과 엘라스틴의 생성을 촉진시키기 위해서는 피부 속 콜라겐과 엘라스틴의 생성을 촉진시키는 것이 중요하다.
피부 속 콜라겐과 엘라스틴의 생성을 촉진시키기 위해서는 피부 속 콜라겐과 엘라스틴의 생성을 촉진시키는 것이 중요하다.
피부 속 콜라겐과 엘라스틴의 생성을 촉진시키기 위해서는 피부 속 콜라겐과 엘라스틴의 생성을 촉진시키는 것이 중요하다.
피부 속 콜라겐과 엘라스틴의 생성을 촉진시키기 위해서는 피부 속 콜라겐과 엘라스틴의 생성을 촉진시키는 것이 중요하다.

Mode seeking

- 최근 연구에 따르면 mode-seeking 방법론에서 생성하는 text는 high-quality가 아님
 - Low ppl != high quality text(human-like)
 - Translation에서 가장 probabl한 string은 empty string
 - [On NMT Search Errors and Model Errors: Cat Got Your Tongue?](#). Stahlberg and Byrne, 2019
 - [Is MAP Decoding All You Need? The Inadequacy of the Mode in Neural Machine Translation](#). Eikema and Aziz, 2020
- Maximization 기법은 dull하고 generic한 text를 생성하고 심지어 Text Degeneration으로 이어짐
 - [Hierarchical Neural Story Generation](#). Fan et al., 2018
 - [The Curious Case of Neural Text Degeneration](#). Holtzman et al., 2020



Multinomial sampling

- 앞서 언급한 바와 같이 Maximization 기법은 dull하고 repetitive한 text를 생성합니다.
- 때문에 [stochastic decoding](#) 전략이 open-ended nlg task에서 많이 사용됩니다.
 - Huggingface에서 Multinomial sampling이라고 이름을 통일
- Sampling 기법은 Maximiazation 기법의 문제점(dull, generic text 생성)을 완화시켜 줍니다.
- 그러나 Incoherent하거나 주제와 관련없는 말을 내뱉곤 하죠. ([unreliable tail](#))
 - Temperature Sampling, Top-K Sampling, Nucleus Sampling 등 이러한 문제를 해결하기 위해 다양한 sampling 전략이 제안됐지만 완벽하게 해결하진 못했습니다.

```
sample_outputs = model.generate(  
    input_ids,  
    do_sample=True,  
    max_length=128,  
    top_p=0.95,  
    top_k=50,  
    num_return_sequences=3,  
)
```

```
print("Output:\n" + 100 * '-')  
for i, sample_output in enumerate(sample_outputs):  
    decoded_text = tokenizer.decode(sample_output, skip_special_tokens=True)  
    print(f"{i}: {decoded_text}", end="\n\n")
```

Output:

0: 근육이 커지기 위해서는 비타민 c가 매우 중요한데, 이 비타민c가 뇌를 건강하게 만드는 데 필요한 핵심 영양
또한 아연이 많이 함유된 음식을 섭취할 경우 심장병이 위험할 수 있으므로, 음식을 씹는 횟수를 줄이도록 하고
또한 비타민 c를 많이 함유한 식품은 스트레스를 해소하고 다이어트에도 도움이 되며, 뇌졸중을 예방하는 효과도
이런 이유로 최근에는 ‘건강한 우리 몸’에서 탄수화물과 지방, 단백질과 비타민을 조합하는 발효식품이 관심을
당근과 생강은 혈중 비타민을 보충해 심혈관계에 도움을

Information-Theoretic View of Natural Language

Information-Theoretic View of Natural Language

- 언어는 인간 의사소통의 주요 수단으로 사용됩니다!
- 이에 대해 정보 이론은 자연어를 연구할 수 있는 직관적인 view를 제공합니다
- 지난 세기 동안 Information Theoretic framework를 사용하여 언어의 발달과 사용에 대한 많은 insight가 만들어졌습니다
 - [A Probabilistic Earley Parser as a Psycholinguistic Model](#). Hale, 2001
 - [The Smooth Signal Redundancy Hypothesis: A Functional Explanation for Relationships between Redundancy, Prosodic Prominence, and Duration in Spontaneous Speech](#). Aylett and Turk, 2004
 - [Information Density and Dependency Length as Complementary Cognitive Models](#). Collins, 2014
 - [Word lengths are optimized for efficient communication](#). Piantadosi et al., 2011
 - [Communicative efficiency, Uniform Information Density, and the Rational Speech Act Theory](#). Levy, 2018
 - [How Efficiency Shapes Human Language](#). Gibson et al., 2019
- 위 작업을 기반으로 정보 이론이 NLG의 특정 동작을 이해하는데 어떻게 도움이 되는지
- 그리고 어떻게 Human-like한 Text를 생성할 수 있는지 알아보시다!

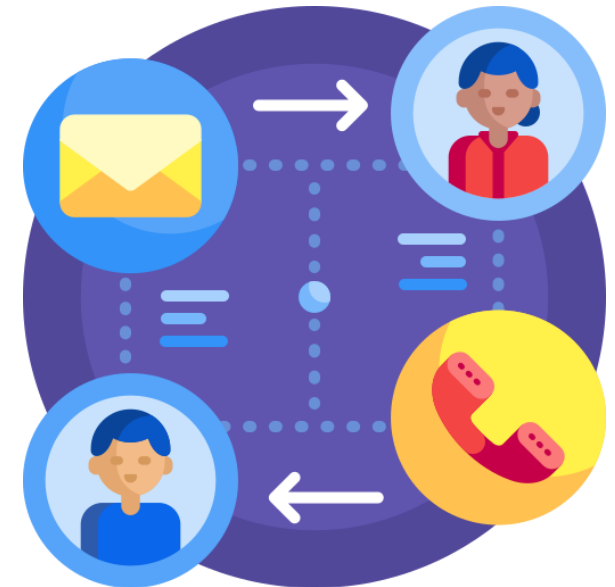
Information-Theoretic View of Natural Language

- Human-like Text를 생성하기 위해서는 사람이 어떻게 자연어를 사용하는지 알아야 한다!
- 그 근거로 Communication, Typicality, UID, RSA 등의 이론을 제시
- 이론을 근거로 새로운 Decoding 전략인 Typical sampling을 제안



The Human Communication Channel

- 자연어 문자열 y 는 정보를 전달하는 수단! 이는 단어(혹은 토큰) y_t 으로 구성됨
- 정보 이론은 y 의 information content가 negative log likelihood로 측정될 수 있음을 말해줌
 - $I(y) := -\log p(y) = \sum_{t=1}^{|y|} I(y_t)$
 - $I(y_t) := -\log p(y_t | y_{<t})$
- 의사소통에서 Agent의 목표는?
 - 정보를 효율적으로 전송
 - 잘못된 의사 소통의 위험을 최소화
- 우리가 전송하고자 하는 정보를 어떻게 인코딩하는지
- 사용하는 단어 및 문법 등의 요구사항을 따를지 말지
 - [How Efficiency Shapes Human Language](#). Gibson et al., 2019

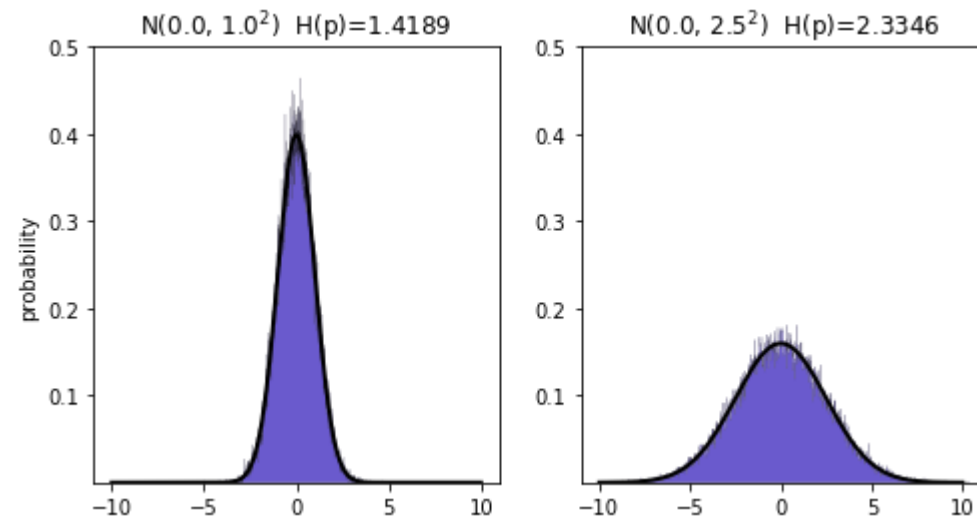


The Human Communication Channel

- Message의 특정 symbol y_t 에 포함되는 expected information content

$$\mathbb{E}[\mathbf{I}(y_t)] = - \sum_{y \in \mathcal{V}} p(y|y_{<t}) \log p(y|y_{<t})$$

- Simply, $H(p(\cdot | y_{<t}))$: Conditional entropy of $p(\cdot | y_{<t})$
- Entropy $\rightarrow$ peaky하면 낮고 flat하면 높음
 - Peaky하다
 - peak한 지점의 값이 나올 확률이 높다
 - 보던 것을 계속 보니 덜 놀랍다 $\rightarrow$ Entropy가 낮다
 - Flat하다
 - 드문 token이 나올 확률이 상대적으로 높다
 - 놀라운 정보를 얻을 가능성이 높다 $\rightarrow$ Entropy가 높다



The Human Communication Channel

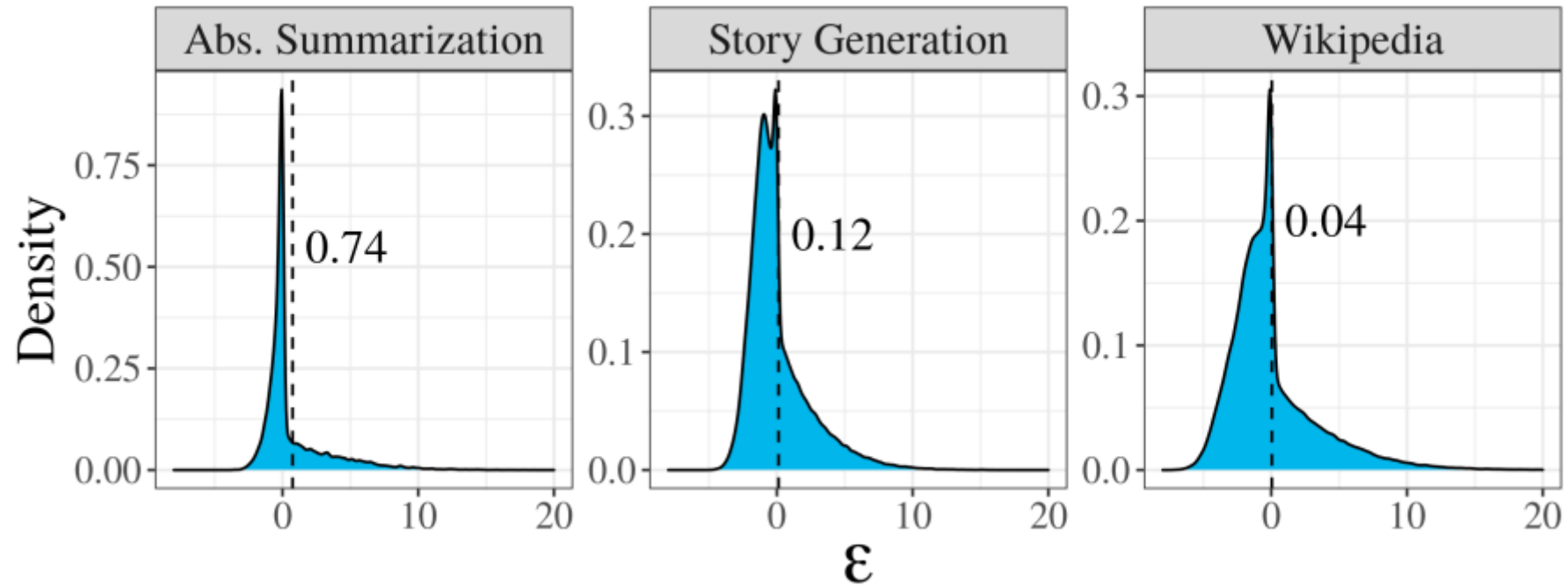
| Hypothesis 3.1.

- Any given word should have an information content close to the expected information content, i.e., the conditional entropy given prior context
 - 주어진 단어는 예상되는 정보 내용(이전 context에서 주어진 조건부 엔트로피)에 가까운 정보 내용을 가져야 함
- In other words, we expect the difference to be small in human-like text for all words y_t
 - 아래 수식에서 우변에서 H 는 expected information content, I 는 단어가 가지는 information content
 - 이 차이를 입실론으로 정의하고 있기 때문에 이 값이 작을수록 위 가설에 맞는 것

$$\varepsilon = |H(p(\cdot|y_{<t})) - I(y_t)|$$

- 사람이 커뮤니케이션할 때 얼마만큼의 정보를 전달할지 정하는 것에 대해서는 여전히 연구 주제
 - 가정하기를, 사람은 Expected information과 information content 차이가 작도록 할 것임
-

The Human Communication Channel



Typicality

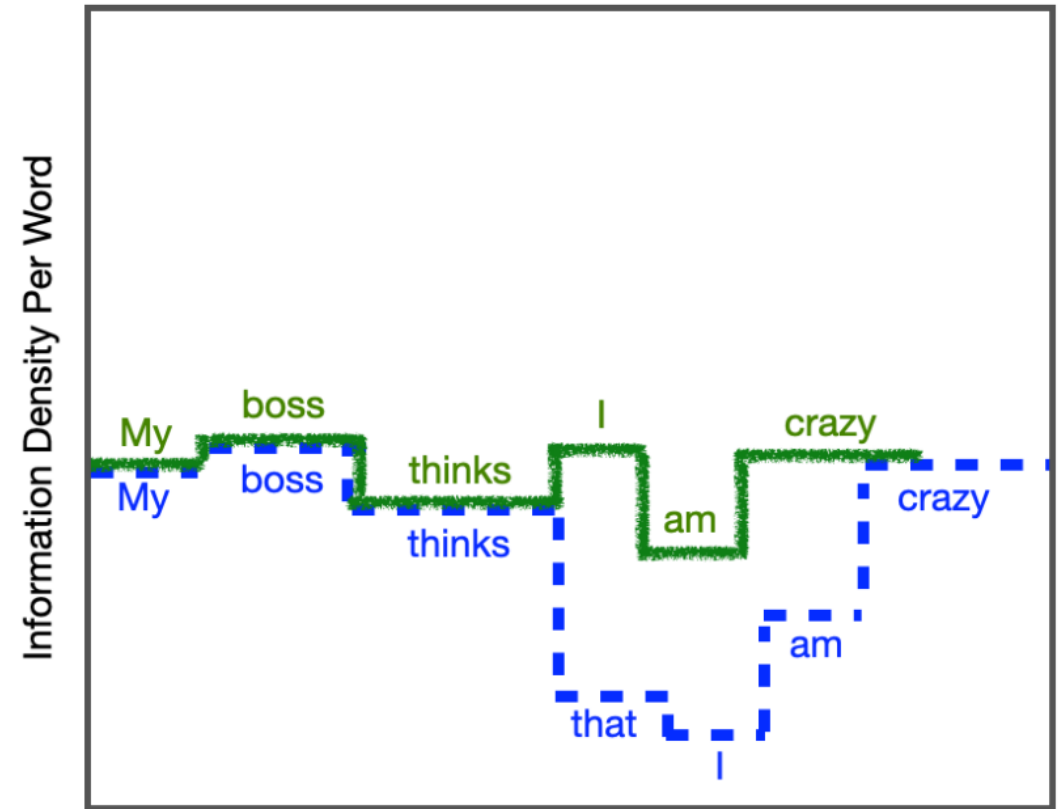
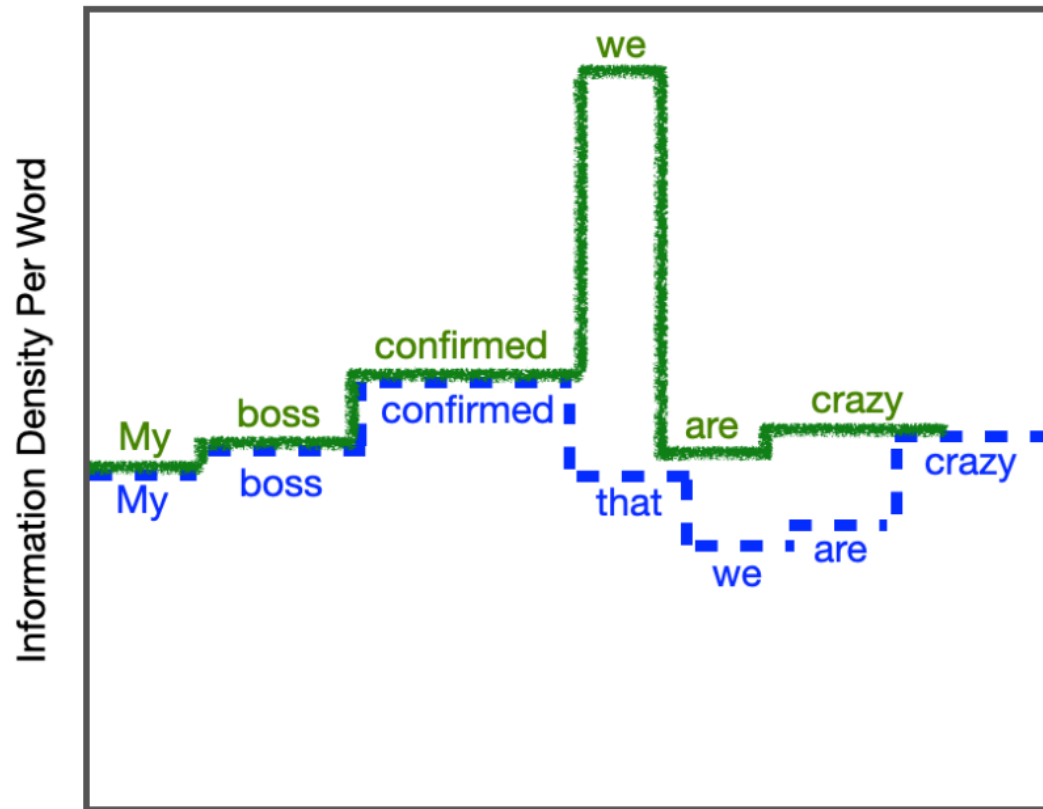
- 논문에선 Stochastic process, Stationary ergodic process 등 deep한 내용을 언급합니다...
- 중요한 부분? 아래는 ϵ -typical message of length N from a stochastic process defined by the distribution p

$$2^{-N(H(p)+\epsilon)} \leq p(y_1, \dots, y_N) \leq 2^{-N(H(p)-\epsilon)}$$

- 위 정의는 가장 높은 확률을 가지는 message가 종종 일반적인 메시지가 아님을 시사!
- 자연어는 모든 곳에서 일관적이어야 한다 (Sounding language should be typical everywhere)

Psycholinguistics: Uniform Information Density

- UID 가설: 화자가 자신의 발화를 구성하여 정보가 각 토큰에 균일하게 배포됨



Psycholinguistics: Uniform Information Density

전문 용어 남발
하던 말 또하고
지루한 얘기

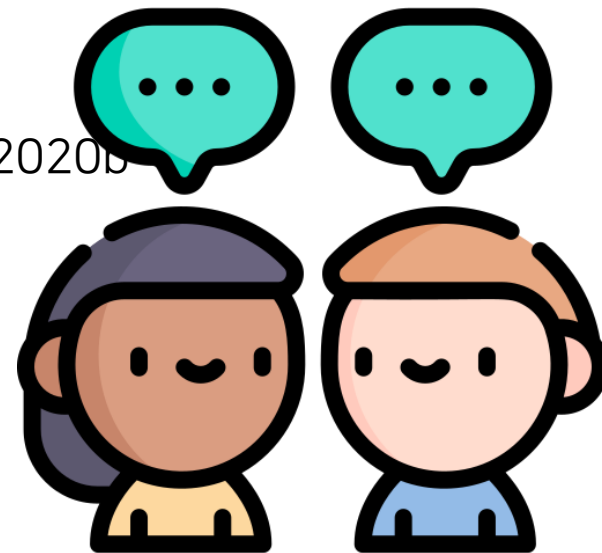


맥락에 맞지 않음

Then, How?

Psycholinguistics: Uniform Information Density

- 한 단어가 Channel이 허용하는 것보다 더 많은 정보를 전달하는 경우 잘못된 의사소통의 위험이 있음
- 동시에 단어가 매우 낮은 정보를 전달하는 경우 이 채널은 비효율적으로 사용됨
- 즉, 화자가 비정상적인 정보 내용을 포함하는 단어를 생성하는 것을 피하며 매 순간에 예상되는 정보에 가까운 단어를 선택하도록 해야한다!
- UID-Inspired objective는 언어 모델의 성능을 증진시킴
 - [A Cognitive Regularizer for Language Modeling](#), Wei et al., 2021
- UID 가설은 기계 번역에서 Beam Search의 효율성에 대한 근거로 사용됨
 - [If beam search is the answer, what was the question?](#). Meister et al., 2020b



Psycholinguistics: Rational Speech Act

- 화용론 → 소통 시 발화에 대한 이론
- Listener와 Speaker는 합리적으로 혹은 실용적으로(Rational) 듣거나 발화
- 다른 쪽의 맥락을 고려하여 그들이 대화하는 context를 기반으로 communication channel을 maximize
- 화자는 단어 당 정보 내용이 Listener가 기대하는 것과 유사한 문자열을 생성함!!

Literal speaker

$$S_{lit}(msg|state) = \frac{\exp(\alpha(\log[msg, state] - C(msg)))}{\sum_{msg'} \exp(\alpha(\log[msg', state] - C(msg'))))}$$

Pragmatic speaker

$$S_{prag}(msg|state) = \frac{\exp(\alpha(\log L_{prag}(state|msg) - C(msg)))}{\sum_{msg'} \exp(\alpha(\log L_{prag}(state|msg') - C(msg'))))}$$

Psycholinguistics: Rational Speech Act

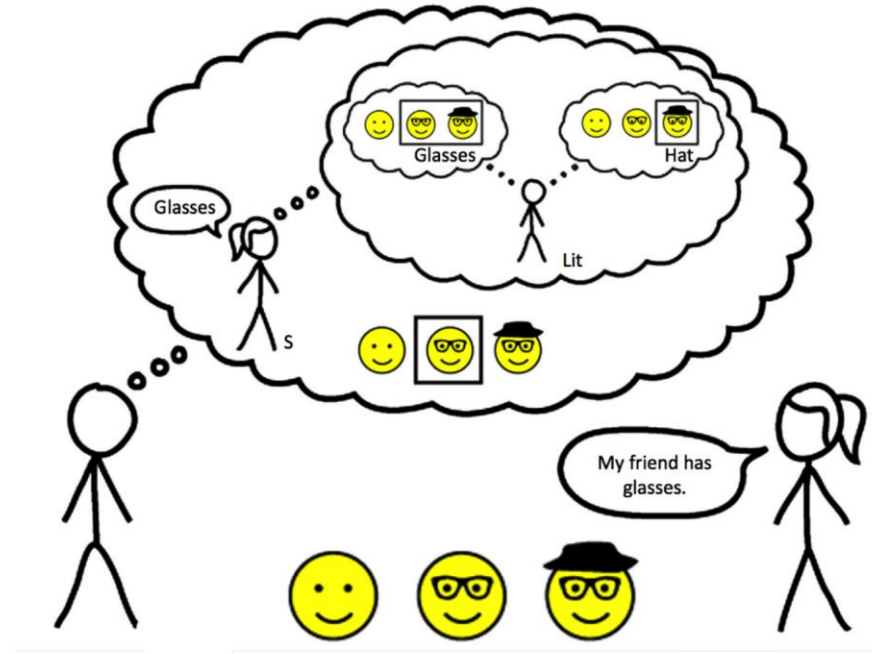
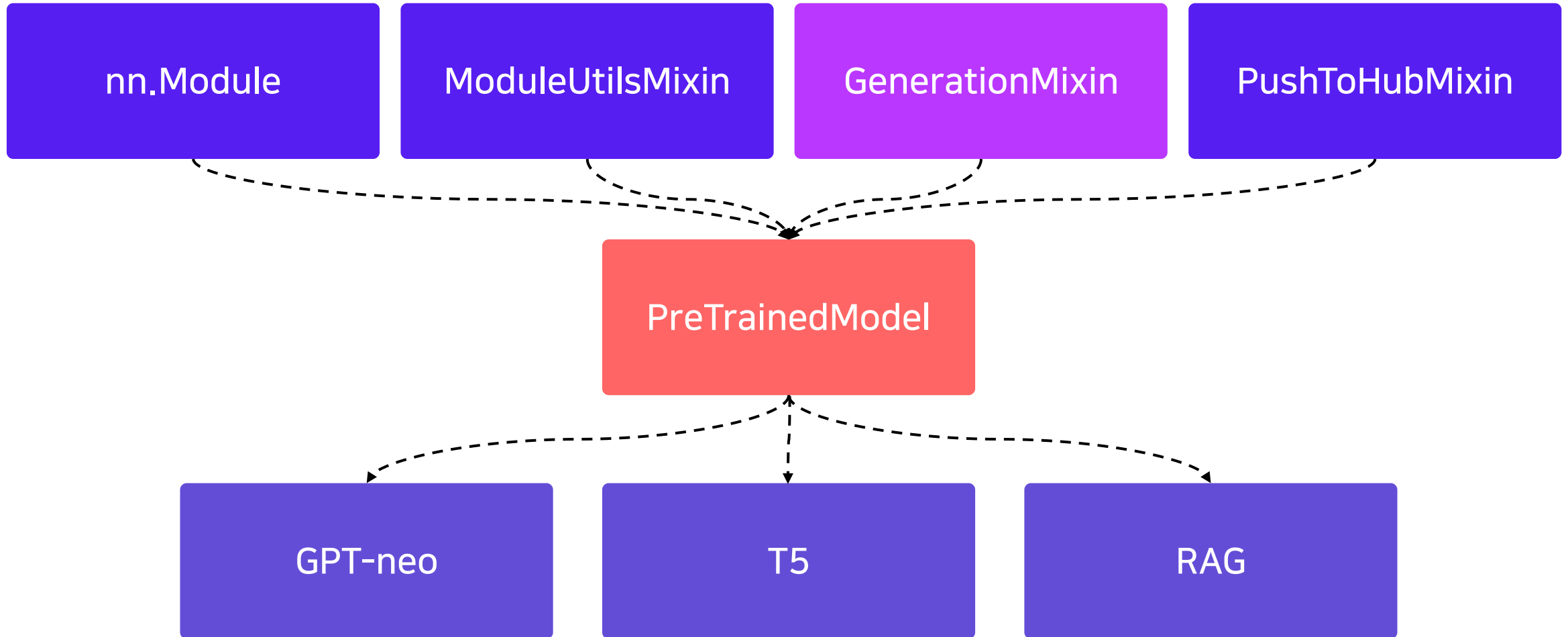


Figure 1: Pragmatic reasoning in a reference game. The speaker says my friend has glasses. The listener needs to guess which one is the speaker's friend. Adapted from Goodman and Frank (2016).

Typical Decoding

generate() method



generate() method

- Set generation parameters
- Define model inputs (seq2seq vs decoder-only) and model kwargs
- Prepare `input_ids` and `max_length`
- Determine generation mode (mode-seeking vs sampling)
- Prepare `distribution pre-processing samplers`
- Prepare stopping criteria
- If multinomial sampling,
 - Prepare `Logits Warper`

generate() method

- Set generation parameters
- Define model inputs (seq2seq vs decoder-only) and model kwargs
- Prepare `input_ids` and `max_length`
- Determine generation mode (mode-seeking vs sampling)
- Prepare `distribution pre-processing samplers`
 - `LM head의 scores를 modify하는데 사용!`
 - `HammingDiversityLogitsProcessor`
 - `RepetitionNGramLogitsProcessor`
 - `EncoderNoRepeatNGramLogitsProcessor`
 - `NoBadWordLogitsProcessor`
 - `MinLengthLogitsProcessor`
 - `PrefixConstrainedLogitsProcessor`
 - `ForcedBOSTokenLogitsProcessor`
 - `ForcedEOSTokenLogitsProcessor`
 - `InfNaNRemoveLogitsProcessor`
 - `ExponentialDecayLengthPenalty`
 - `CustomProcessor`
 - `LogitNormalization`
- Prepare stopping criteria
- If multinomial sampling,
 - Prepare `Logits Warper`
- Beam Search는 따로 구현체를 보시는 것을 추천합니다!
→ Dull한 text 말고 randomness를 부여!

LogitsWarper?

- <https://github.com/huggingface/transformers/pull/6949>

Refactoring the generate() function #6949

 Merged patrickvonplaten merged 52 commits into `huggingface:master` from `patrickvonplaten:generation_refactor`  on 4 Nov 2020

 Conversation 84

 Commits 52

 Checks 0

 Files changed 38



patrickvonplaten commented on 5 Sep 2020 • edited ▼

Member



Generation refactor

This is a possible design that IMO would make generate much more readable and flexible for future code changes. The code shown in this PR is more or less pseudo code, but I'm sure that this design is fully backwards compatible (for the `generate()` function, I don't plan on keep `beam_search_generation` and `_no_beam_search_generation` -> I don't think anybody directly accessed these functions).

It's probably better to look at the code directly, here:

https://github.com/huggingface/transformers/blob/3c8a12a3bf44aa7845675852e28c47d0a9cb808e/src/transformers/generation_utils.py then at the diff.

The following major changes are made:

LogitsWarper?

- <https://github.com/huggingface/transformers/issues/4164>

Add a `sampling_transform` callback to generation for arbitrary probability-warps #4164

New issue

Closed

turtlesoupy opened this issue on 6 May 2020 · 7 comments



turtlesoupy commented on 6 May 2020



Feature request

I'd like to add `sampling_transform` callback argument to all `generate` functions in `modeling_utils` that allows arbitrary sampling from the probability distribution during sequence generation. The signature of this function would be `(input_ids, next_probs, next_token) -> next_token`.

Motivation

The `modeling_utils`'s generation function is getting pretty hairy -- including parameters for top p, top k, bad tokens, temperature, repetition penalty, etc.. Every new way of sampling means that we have to add more parameters and it further complicate the function. I believe the right way of solving this is to provide a function-argument that allows users to express arbitrary rules for the next sample. In the long run, we could replace all the other parameters with a set of pre-baked `sampling_transform` functions that can be composed at will.

This method scales better to strange warps -- for example, in a project I'm working on (<https://github.com/turtlesoupy/this-word-does-not-exist>) I need to early-terminate sequences if they generate from a large set of bad tokens and need to continue generating if an EOS token is sampled too early. An example is here https://github.com/turtlesoupy/this-word-does-not-exist/blob/260e33a8f420b9be8b1e7260cb03c74d6231686e/title_maker_pro/datasets.py#L386

Assignees

patrickvonplaten

Labels

wontfix

Projects

None yet

Milestone

No milestone

Development

No branches or pull requests

Notifications

Customize

Subscribe

LogitsWarper?

- https://github.com/huggingface/transformers/blob/main/src/transformers/generation_logits_process.py

```
class LogitsProcessor:
    """Abstract base class for all logit processors that can be applied during generation."""

    @add_start_docstrings(LOGITS_PROCESSOR_INPUTS_DOCSTRING)
    def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor) -> torch.FloatTensor:
        """Torch method for processing logits."""
        raise NotImplementedError(
            f"{self.__class__} is an abstract class. Only classes inheriting this class can be called."
        )

class LogitsWarper:
    """Abstract base class for all logit warpers that can be applied during generation with multinomial sampling."""

    @add_start_docstrings(LOGITS_PROCESSOR_INPUTS_DOCSTRING)
    def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor) -> torch.FloatTensor:
        """Torch method for warping logits."""
        raise NotImplementedError(
            f"{self.__class__} is an abstract class. Only classes inheriting this class can be called."
        )
```

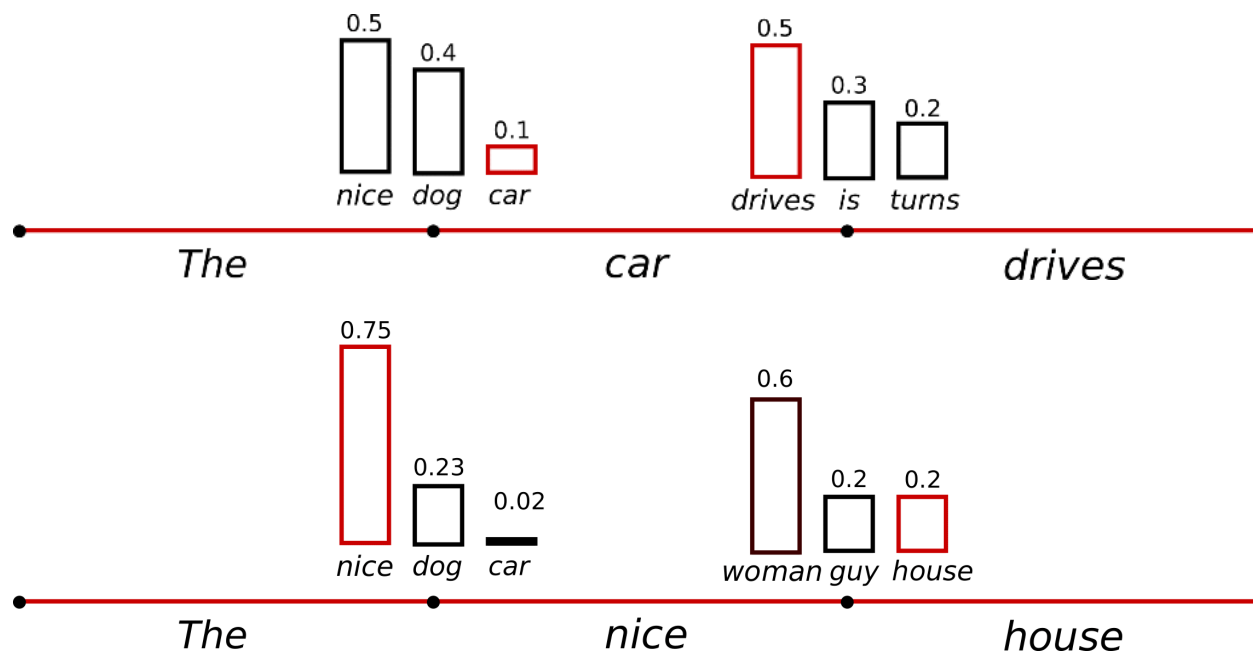
LogitsWarper?

```
# init warp parameters
top_k = top_k if top_k is not None else self.config.top_k
top_p = top_p if top_p is not None else self.config.top_p
typical_p = typical_p if typical_p is not None else self.config.typical_p
temperature = temperature if temperature is not None else self.config.temperature
# instantiate warpers list
warpers = LogitsProcessorList()

# the following idea is largely copied from this PR: https://github.com/huggingface/transformers/pull/5420/files
# all samplers can be found in `generation_utils_samplers.py`
if temperature is not None and temperature != 1.0:
    warpers.append(TemperatureLogitsWarper(temperature))
if top_k is not None and top_k != 0:
    warpers.append(TopKLogitsWarper(top_k=top_k, min_tokens_to_keep=(2 if num_beams > 1 else 1)))
if top_p is not None and top_p < 1.0:
    warpers.append(TopPLogitsWarper(top_p=top_p, min_tokens_to_keep=(2 if num_beams > 1 else 1)))
if typical_p is not None and typical_p < 1.0:
    warpers.append(TypicalLogitsWarper(mass=typical_p, min_tokens_to_keep=(2 if num_beams > 1 else 1)))
# `LogitNormalization` should always be the last logit processor, when present
if renormalize_logits is True:
    warpers.append(LogitNormalization())
return warpers
```

Temperature sampling

- 분포를 Sharp하게 만들기
 - 높은 확률을 가지는 단어의 likelihood를 높이고
 - 낮은 확률을 가지는 단어의 likelihood를 낮추는 것



$$P(x_i|x_{1:i-1}) = \frac{\exp(u_i/t)}{\sum_j \exp(u_j/t)}$$

```
class TemperatureLogitsWarper(LogitsWarper):
    def __init__(self, temperature: float):
        self.temperature = temperature

    def __call__(
        self,
        input_ids: torch.Tensor,
        scores: torch.Tensor
    ) -> torch.FloatTensor:
        scores = scores / self.temperature
        return scores
```


Various LogitsWarper

- Top-k

$$\pi(y|y_{<t}) = \begin{cases} \frac{q(y|y_{<t})}{\sum_{y \in V^{(k)}} q(y|y_{<t})} & \text{if } y \in V^{(k)} \\ 0, & \text{if o.w} \end{cases}$$

- Top-p

$$\pi(y|y_{<t}) = \begin{cases} \frac{q(y|y_{<t})}{\sum_{y \in V^{(n)}} q(y|y_{<t})} & \text{if } y \in V^{(n)} \\ 0, & \text{if o.w} \end{cases}$$

$$V^{(n)} \text{ s.t. } \sum_{y \in V^{(n)}} q(y|y_{<t}) \geq n$$

- Typical-p

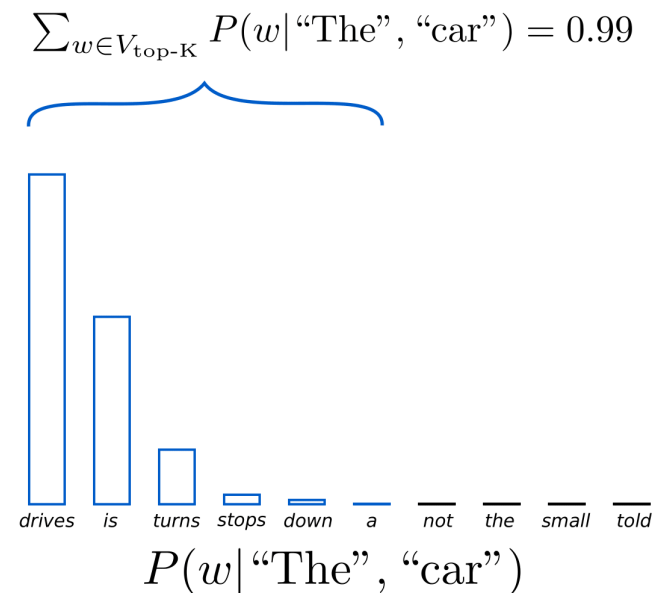
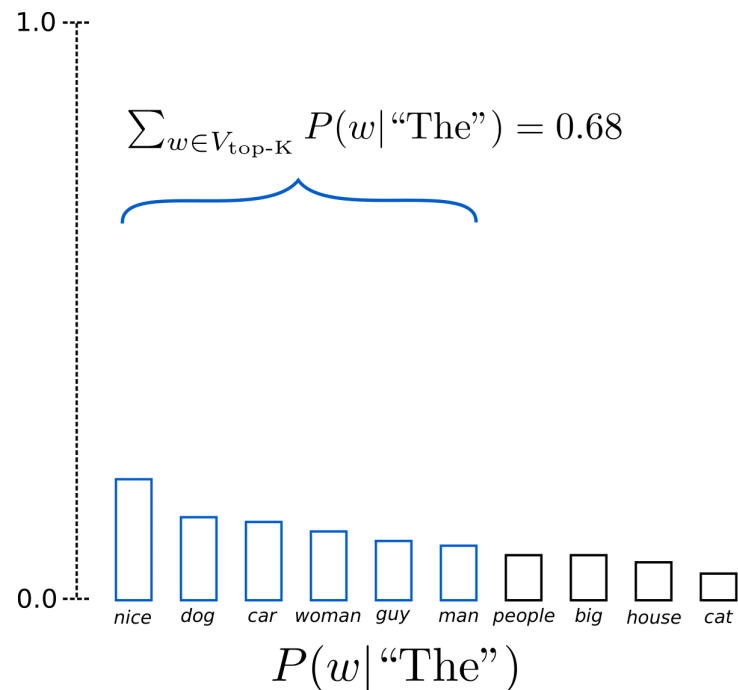
$$\pi(y|y_{<t}) = \begin{cases} \frac{q(y|y_{<t})}{\sum_{y \in V^{(\tau)}} q(y|y_{<t})} & \text{if } y \in V^{(\tau)} \\ 0, & \text{if o.w} \end{cases}$$

$$V^{(\tau)} \text{ s.t. } \min \sum_{y \in V^{(\tau)}} |H(q(\cdot | y_{<t})) + \log q(y|y_{<t})|$$

Top-k sampling

- Decoding 단계에서 가장 가증성이 높은 상위 K개의 단어로 sampling space를 제한

$$\pi(y|y_{<t}) = \begin{cases} q(y|y_{<t})/Z_t, & \text{if } y \in \mathcal{V}^{(k)} \\ 0, & \text{o.w} \end{cases} \quad Z_t = \sum_{y \in \mathcal{V}^{(k)}} q(y|y_{<t}), \quad \mathcal{V}^{(k)} \subseteq \mathcal{V}$$



Top-k sampling

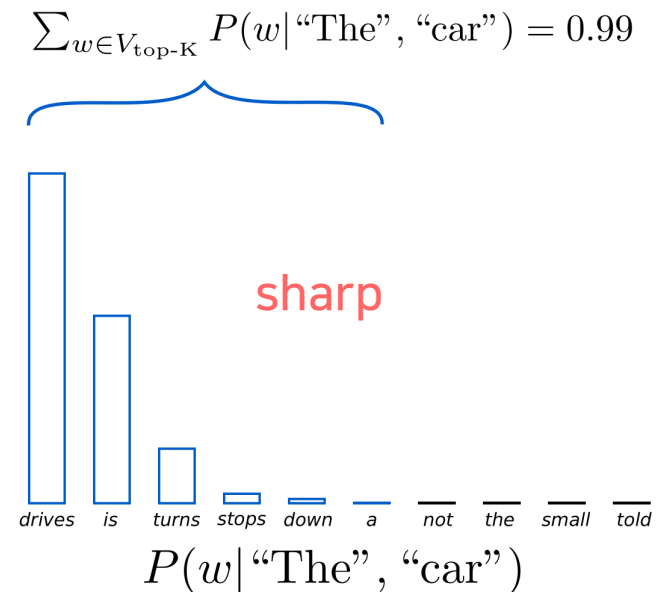
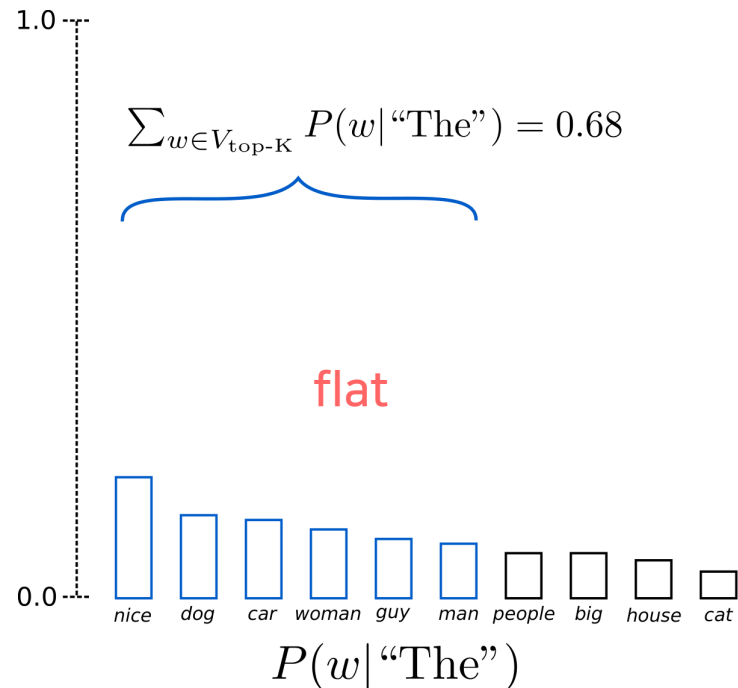
$$\pi(y|y_{<t}) = \begin{cases} q(y|y_{<t})/Z_t, & \text{if } y \in \mathcal{V}^{(k)} \\ 0, & \text{o.w} \end{cases}$$

```
def __call__(
    self,
    input_ids: torch.LongTensor,
    scores: torch.FloatTensor,
) -> torch.FloatTensor:
    top_k = min(max(self.top_k, self.min_tokens_to_keep), scores.size(-1)) # Safety check
    # Remove all tokens with a probability less than the last token of the top-k
    indices_to_remove = scores < torch.topk(scores, top_k)[0][..., -1, None]
    scores = scores.masked_fill(indices_to_remove, self.filter_value)
    return scores
```

```
class TopKLogitsWarper(LogitsWarper):
    def __init__(
        self,
        top_k: int,
        filter_value: float = -float("Inf"),
        min_tokens_to_keep: int = 1,
    ):
        self.top_k = top_k
        self.filter_value = filter_value
        self.min_tokens_to_keep = min_tokens_to_keep
```

Top-k sampling

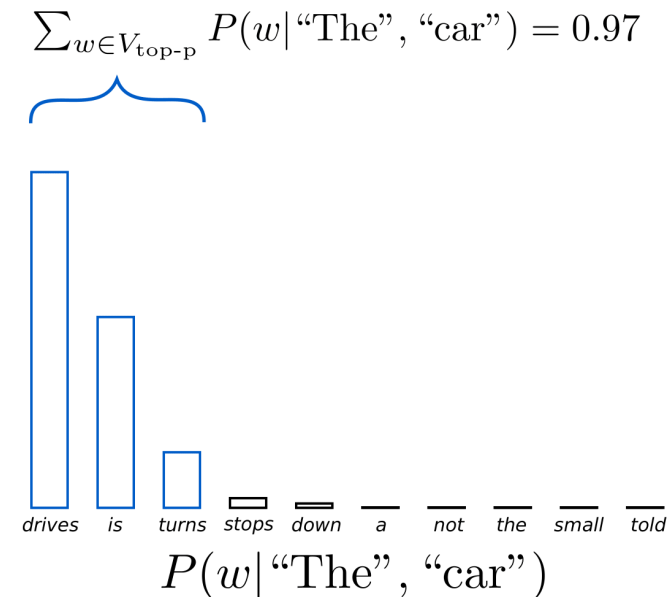
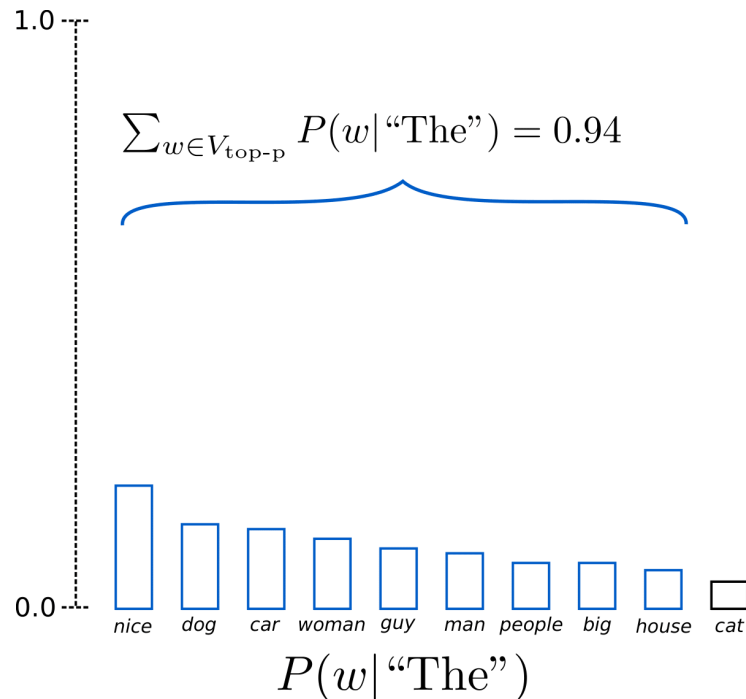
- Top-k → 고정된 K를 사용
- Sample pool을 고정된 크기로 제한하기 때문에 모델이 sharp distribution에 대해 횡설수설(gibberish)할 위험이 있고 flat distribution에 대해 창의성(creativity)이 제한될 수 있음



Top-p sampling (Nucleus sampling)

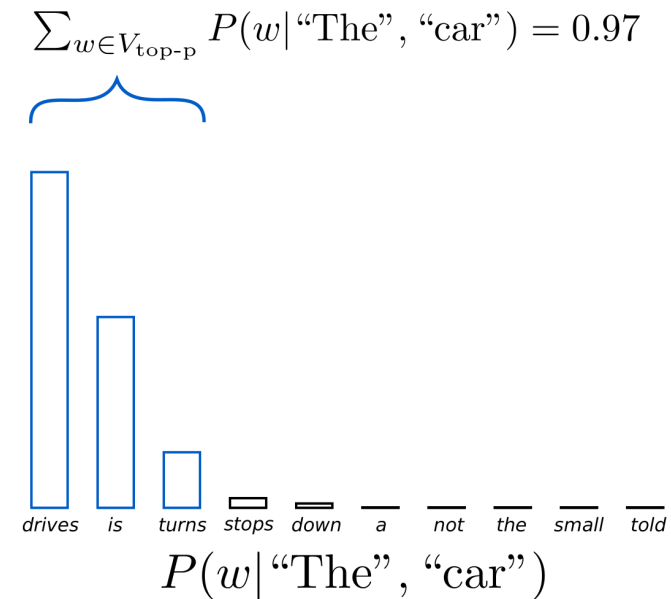
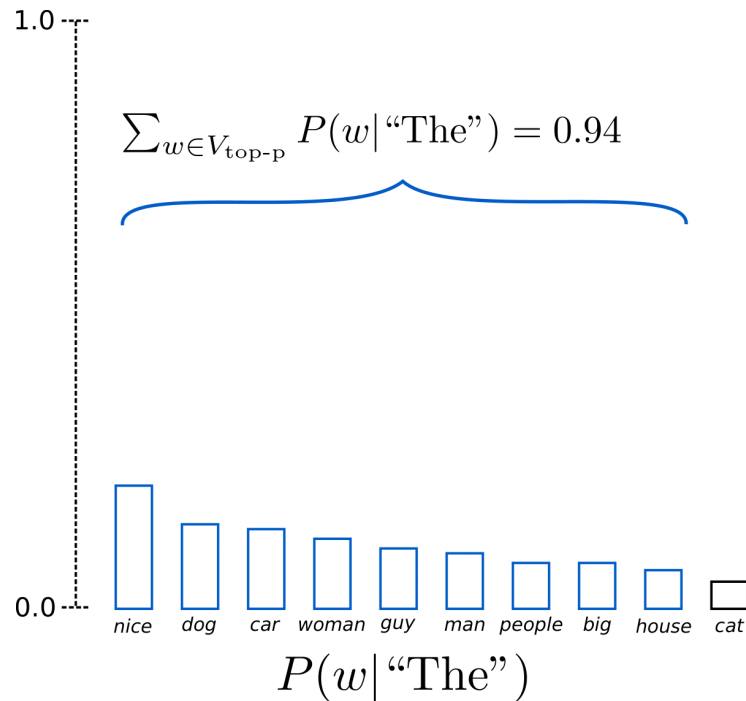
- Cumulative probability mass가 선택된 threshold n 을 초과하는 단어의 가장 작은 nucleus, 즉 subset을 고려

$$\sum_{y \in \mathcal{V}^{(n)}} q(y|y_{<t}) \geq n \quad Z_t = \sum_{y \in \mathcal{V}^{(n)}} q(y|y_{<t}), \quad \mathcal{V}^{(n)} \subseteq \mathcal{V}$$



Top-p sampling (Nucleus sampling)

- Scores로 정렬 및 누적 확률 계산
- 누적 확률이 $n(\text{top_p})$ 보다 작은 tokens들 제거
- Sorted tensors를 original index로 scatter

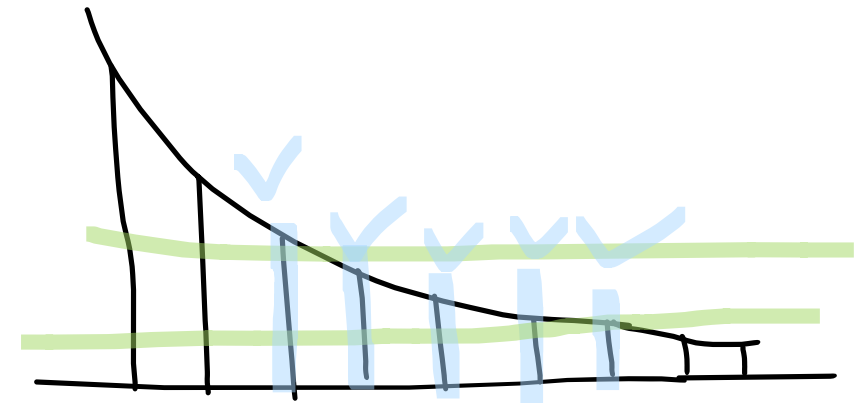


Typical-p sampling (Typical Decoding)

- Sampling distribution을 해당 time step에서 모델의 conditional entropy로부터 특정 absolute 범위 내에서 negative log-likelihood를 갖는 단어로만 제한
- Hyperparameter τ : the amount of probability mass from the original distribution that we wish to consider
- Too obvious하거나 High probability를 갖는 단어보단 expected words를 sampling!

$$\pi(y|y_{<t}) = \begin{cases} \frac{q(y|y_{<t})}{\sum_{y \in V^{(\tau)}} q(y|y_{<t})} & \text{if } y \in V^{(\tau)} \\ 0, & \text{if o.w} \end{cases}$$

$$V^{(\tau)} \text{ s.t. } \min \sum_{y \in V^{(\tau)}} |H(q(\cdot | y_{<t})) + \log q(y|y_{<t})|$$



Typical-p sampling (Typical Decoding)

- Entropy 계산 후 score $-\log q$ - H 계산
- Scores로 정렬 및 누적 확률 계산
- 누적 확률이 $n(\text{top_p}) \tau(\text{probability mass})$ 보다 작은 tokens들 제거
- Sorted tensors를 original index로 scatter

Top-p와 다른 부분!

$$\mathbb{E}[\mathbf{I}(y_t)] = - \sum_{y \in \mathcal{V}} p(y|y_{<t}) \log p(y|y_{<t})$$

```
# calculate entropy
normalized = torch.nn.functional.log_softmax(scores, dim=-1)
p = torch.exp(normalized)
# inference time이기 때문에 time step마다 entropy를 계산
ent = -(normalized * p).nansum(-1, keepdim=True) # nan -> 0으로 handle
```


Typical-p sampling (Typical Decoding)

- Entropy 계산 후 score $|- \log q - H|$ 계산
- Scores로 정렬 및 누적 확률 계산
- 누적 확률이 $n(\text{top_p}) \tau(\text{probability mass})$ 보다 작은 tokens들 제거
- Sorted tensors를 original index로 scatter

$$\sum_{y \in V^{(\tau)}} |H(q(\cdot | y_{<t})) + \log q(y | y_{<t})|$$

```
# shift and sort
shifted_scores = torch.abs((-normalized) - ent) #  $|- \log q - H|$ 
sorted_scores, sorted_indices = torch.sort(shifted_scores, descending=False)
sorted_logits = scores.gather(-1, sorted_indices)
cumulative_probs = sorted_logits.softmax(dim=-1).cumsum(dim=-1)

# Remove tokens with cumulative mass above the threshold
last_ind = (cumulative_probs < self.mass).sum(dim=1)
last_ind[last_ind < 0] = 0
```

Computational Complexity

- 0| schema는 nucleus 혹은 top-k sampling과 동일 효율로 구현할 수 있음
 1. Compute the conditional entropy: $\mathcal{O}(|\mathcal{V} \log |\mathcal{V}|)$
 2. Sort words by $|H(p(\cdot|y_{<t})) - I(y_t)|$: $\mathcal{O}(|\mathcal{V} \log |\mathcal{V}|)$
 3. Greedily take words from the list until their cumulative prob. exceeds the threshold τ : $\mathcal{O}(|\mathcal{V}|)$

Experiments & Results

Setup: Models and Data, Hyperparameters

- 🤖 transformers의 nucleus, top-k, temperature sampling, beam search 구현체 사용
- Mirostat은 저자 구현체 사용
- Datasets and Models
 - WritingPrompts (Story Generation): GPT-2 (large, medium)
 - WikiText-103: GPT-2 medium
 - CNN/DailyMail(Abstractive Summarization): BART
- MAUVE를 사용하여 sweep
 - Top-k: $k=3, 40$
 - Nucleus: $n=0.9, 0.95$
 - Mirostat: $\tau=3.0$
 - Typical-p: $\tau=0.2, 0.95$

Evaluation

- Automatic Quality Metrics

- Generated texts' perplexity
 - PPL(g): the model used to generate the text
 - PPL(i): an independent LM (external LM)
- MAUVE score
 - LM embedding을 두 text distribution의 유사도를 측정하는데 사용

- Automatic Diversity Metrics

- REP(repetition) → Neural Text Generation With Unlikelihood Training. Welleck et al., 2020
- Zipf's coefficient → Unigram rank-frequency statistics, reference corpus와 얼마나 유사한지
- N-gram diversity → 클수록 Diverse

- Human Evaluations

- Amazon Mechanical Turk 사용하여 task별 그리고 각 decoding strategy 별 200 examples에 대하여 5명의 annotators에 의해 평가된 text quality의 human judgement
- Abstractive Summarization → fluency, relevance
- Story Generation → fluency, coherence, interestingness

Quantitative Performance

	Story Generation						Human ($\uparrow$)
	PPL (g)	PPL (i)	MAUVE ($\uparrow$)	REP ($\downarrow$)	Zipf	D ($\uparrow$)	
Reference	16.33	3.19	—	0.28	1.09	0.85	4.12 (± 0.02)
Temperature ($\tau=0.5$)	25.34 (+9.01)	2.90 (-0.29)	0.95	0.25	1.07 (-0.02)	0.87	4.13 (± 0.02)
Temperature ($\tau=1$)	25.67 (+9.34)	2.39 (-0.8)	0.95	0.26	1.07 (-0.02)	0.87	4.13 (± 0.02)
Nucleus ($n=0.9$)	7.75 (-8.58)	2.23 (-0.96)	0.95	0.35	1.29 (+0.20)	0.79	4.09 (± 0.02)
Nucleus ($n=0.95$)	11.65 (-4.68)	2.39 (-0.80)	0.95	0.30	1.20 (+0.11)	0.84	4.13 (± 0.02)
Top- k ($k=30$)	7.07 (-9.26)	2.90 (-0.29)	0.88	0.35	1.41 (+0.32)	0.80	4.13 (± 0.02)
Top- k ($k=40$)	11.83 (-4.5)	2.50 (-0.69)	0.92	0.35	1.33 (+0.24)	0.82	4.09 (± 0.02)
Mirostat ($\tau=3$)	8.14 (-8.19)	3.13 (-0.06)	0.93	0.34	1.30 (+0.21)	0.83	4.12 (± 0.02)
Typical ($\tau=0.2$)	14.25 (-2.08)	3.13 (-0.06)	0.78	0.30	1.27 (+0.18)	0.84	4.15 (± 0.02)
Typical ($\tau=0.95$)	11.59 (-4.74)	2.39 (-0.80)	0.96	0.31	1.21 (+0.12)	0.84	4.13 (± 0.02)

Table 1: Automatic quality and diversity metrics, as described in Section 5.1, along with human ratings on the WRITINGPROMPTS dataset. Human ratings are averaged across criteria to form a single metric. Bolded values are the best results among decoding strategies, where for perplexity and Zipf’s coefficient, we take this to be the delta from measurements on human text (numbers in purple). Numbers in blue are standard error estimates. Results are from fine-tuned GPT-2 large; results for GPT-2 medium are given in Appendix B.

Quantitative Performance

	Abstractive Summarization						Human ($\uparrow$)
	PPL (g)	PPL (i)	MAUVE ($\uparrow$)	REP ($\downarrow$)	Zipf	D ($\uparrow$)	
Reference	10.29	3.35	—	0.13	0.76	0.97	4.31 (± 0.03)
Beam ($k=5$)	1.39 (-8.90)	3.35 ($+0.00$)	0.90	0.14	0.77 ($+0.01$)	0.97	4.35 (± 0.03)
Temperature ($\tau=0.5$)	7.10 (-3.19)	3.73 ($+0.38$)	0.97	0.15	0.75 (-0.01)	0.97	4.25 (± 0.03)
Temperature ($\tau=1$)	6.46 (-3.83)	3.35 ($+0.00$)	0.95	0.14	0.75 (-0.01)	0.97	4.29 (± 0.03)
Nucleus ($n=0.9$)	2.97 (-7.32)	3.27 (-0.08)	0.90	0.17	0.93 ($+0.17$)	0.96	4.26 (± 0.03)
Nucleus ($n=0.95$)	3.96 (-6.33)	3.64 ($+0.29$)	0.99	0.15	0.91 ($+0.15$)	0.97	4.26 (± 0.03)
Top- k ($k=30$)	3.13 (-7.16)	3.27 (-0.08)	0.98	0.16	0.93 ($+0.17$)	0.97	4.31 (± 0.03)
Top- k ($k=40$)	3.26 (-7.03)	3.12 (-0.23)	0.96	0.16	0.93 ($+0.17$)	0.97	4.29 (± 0.03)
Typical ($\tau=0.2$)	3.80 (-6.49)	3.79 ($+0.44$)	0.72	0.14	0.91 ($+0.15$)	0.97	4.27 (± 0.03)
Typical ($\tau=0.95$)	3.86 (-6.43)	3.64 ($+0.29$)	0.96	0.15	0.92 ($+0.16$)	0.97	4.32 (± 0.03)

Table 2: Automatic quality and diversity metrics, as described in Section 5.1, along with human ratings on the CNN/DAILYMAIL dataset. Human ratings are averaged across criteria to form a single metric. Bolded values are the best results among decoding strategies, where for perplexity and Zipf’s coefficient, we take this to be the delta from measurements on human text (numbers in purple). Numbers in blue are standard error estimates.

Quantitative Performance

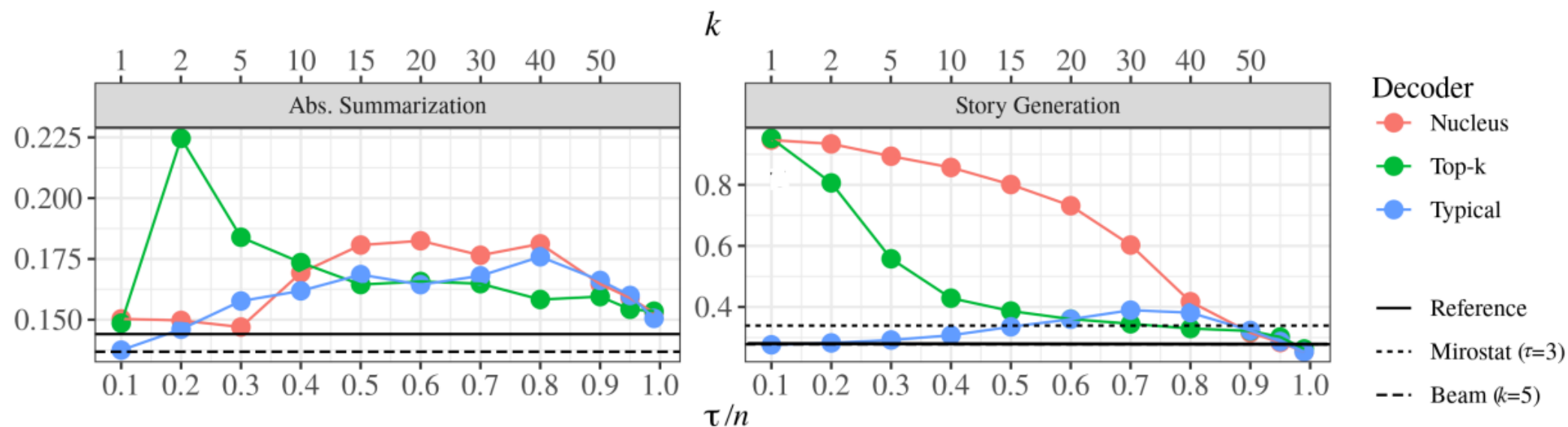


Figure 2: REP (Welleck et al., 2020) values for different k and τ/n (lower is better). Lines indicate REP measurement for reference text and Mirostat/beam search.

Qualitative Performance

- For the **abstractive summarization** task,
 - Typical sampling이 Beam Search와 매우 유사하게 포괄적이고 일관된 요약을 제공
 - Temperature sampling의 경우 일관성(coherent)이 떨어짐
 - Nuclues, Top-K sampling의 text는 기사의 일부 중요한 정보(e.g., 절도 및 방화 혐의)를 놓치고 있음
- For the **story generation** task,
 - 요약보다 훨씬 주관적인 task
 - Typical sampling은 모든 decoding strategy 중 가장 유창한 story를 제공
 - 다른 이야기는 일관성이 부족하고 처음 몇 문장 내에서도 반복되는 구문과 단어를 볼 수 있음

Conclusion

Typical Decoding



x **ETH** zürich

- Language model $\approx$ probability distributions that defines a communication channel
 - 모델에서 생성된 text는 위 channel에서 추출된 message와 같다
- 심리 언어학 및 화용론(pragmatics)의 결과에 Motivated된 논문
- Assumption
 - 인간은 효율적인 의사 소통을 목표로 한다
 - 즉, 단어 당 정보 내용이 사전 context에서 주어진 단어의 예상 정보 내용과 가까운 범위 내에 있는 text를 생성한다. (그 대화 맥락에 맞는 단어들)
- 이를 기반으로 Typical Decoding이라는 새로운 multinomial sampling 디코딩 전략을 제안
- 추상 요약과 스토리 생성 task에서 typical sampling이 잘한다는 사실을 보임
- 그러나 정량지표로는 큰 개선을 느끼긴 어려움
- 올바른 방향이고 흥미롭지만 quantitative analysis를 보고 납득하긴 어렵다!
- Huggingface에 구현되어 있으니 새로운 parameter를 많이 써보고 경험을 community에 공유하자!

Any Questions?